DSC 40B

Theoretical Foundations II

Lecture 5 | Part 1

Lower Bound Theory

Today in DSC 40B...

- Finish Lecture 04: theoretical lower bounds.
  - TLBs are covered by Quiz 02.
- How do we analyze the time complexity of **recursive** algorithms?
- How do we know that our recursive code is **correct**?

Imagine...

- You write a simple algorithm to solve a problem.
- You analyze time complexity and find it is $\Theta(n^2)$.
- You ask yourself: *can I do better than* $\Theta(n^2)$*?*
- Or: *What is the best time complexity possible?*

Doing Better

- How can you know what you don't know?
- You can argue that *any* algorithm for solving the problem *must* take at least a certain amount of time in the worst case.

Example: Minimum

- Problem: Find minimum in array of length n.
- *Any* algorithm has to check all n numbers in the worst case.
  - Or else the number not checked could have been the smallest!
- Takes at least linear ($\Omega(n)$) time.
  - **No algorithm** for the min can have worst case of < linear time.

Definition

A **theoretical lower bound** is a lower bound on the **worst-case** time complexity of **any algorithm** solving a particular problem.

Main Idea

No algorithm's worst case can possibly be better than theoretical lower bound.

Speed Limit

- A theoretical lower bound is a **speed limit**.
- No algorithm can be faster than the TLB.

Loose Lower Bounds

- $\Omega(\log n)$, $\Theta(\sqrt{n})$ and $\Theta(1)$ are also theoretical lower bounds for finding the minimum.
- But no algorithm can exist which has a worst case of $\Theta(\log n)$, $\Theta(\sqrt{n})$, or $\Theta(1)$.
- This bound is **loose**. Not super useful.

Tight Lower Bounds

- A lower bound is **tight** if there exists an algorithm with that worst case time complexity.
- That algorithm is (in a sense) **optimal**.

Definition

A **tight theoretical lower bound** for a problem is the **fastest** possible worst-case time complexity of any algorithm solving that problem.

How to find a TLB

- Argument from completeness:
  - The algorithm might not be correct if it doesn't check k things, so the time is $\Omega(k)$.
- Argument from I/O:
  - If the output is an array of size k, time taken is $\Omega(k)$
- More sophisticated arguments...

Tight Bounds can be difficult to find

- Often require sophisticated combinatorial arguments outside of the scope of DSC 40B.

Assumptions make problems easier

- The TLB for finding a minimum changes if we assume that the array is sorted.

Exercise

Consider these two problems:

1. Find the min of a sorted array.
2. Given a target t and a sorted array, determine whether t is in the array.

Find tight theoretical lower bounds for each problem.

Main Idea

When coming up with an algorithm, first try to find a tight TLB. Then try to make an algorithm which has that worst-case complexity. If you can, it's **optimal**!

Practice makes perfect

- `dsc40b.com/practice` has a dozen more examples of finding theoretical lower bounds.

DSC 40B

Theoretical Foundations II

Lecture 5 | Part 2

Case Study: Matrix Multiplication

It's Important

- Matrix multiplication is a *very* common operation in machine learning algorithms.
- **Estimate**: 75% - 95% of time training a neural network is spent in matrix multiplication.

Recall

- If A is $m \times p$ and B is $p \times n$, then AB is $m \times n$.
- The ij entry of AB is

$$(AB)_{ij} = \sum_{k=1}^{p} a_{ik} b_{kj}$$

Recall

$$(AB)_{ij} = \sum_{k=1}^{p} a_{ik} b_{kj}$$

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \begin{pmatrix} 5 & -1 \\ 1 & 7 \\ -2 & -3 \end{pmatrix} = \begin{pmatrix} & \\ & \\ & \end{pmatrix}$$

Naïve Algorithm

- This algorithm is relatively straightforward to code up.

```
def mmul(A, B):
    """
    A is (m x p) and B is (p x n)
    """
    m, p = A.shape
    n = B.shape[1]

    C = np.zeros((m, n))

    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i,j] += A[i,k] * B[k, j]

    return C
```

Time Complexity

- The naïve algorithm takes time $\Theta(mnp)$.
- If both matrices are $n \times n$, then $\Theta(n^3)$ time.
- **Cubic!**

Cubic Time Complexity

- The largest problem size that can be solved, if a basic operation takes 1 nanosecond.

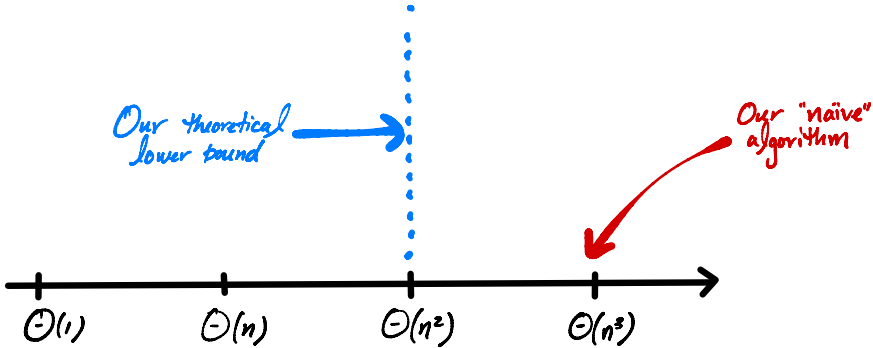
1 s	10 m	1 hr
1,000	6,694	15,326

The Question

- Can we do better?
- How fast can we possibly multiply matrices?

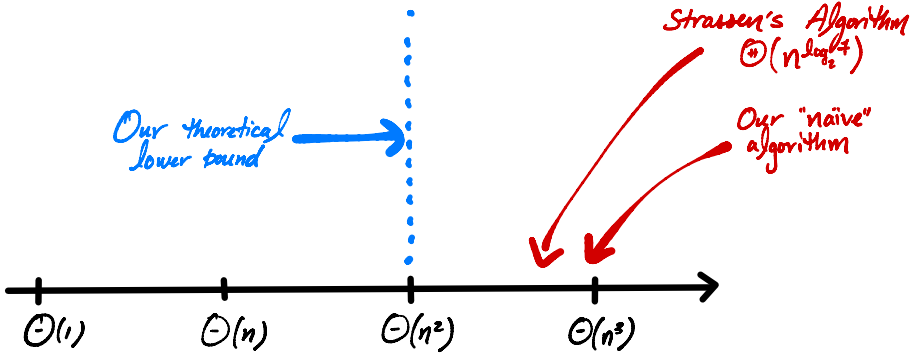
Theoretical Lower Bound

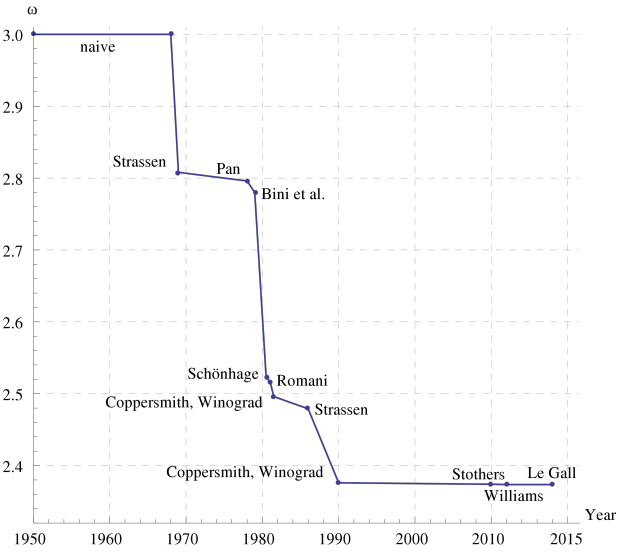
- If A and B are $n \times n$, C will have n^2 entries.
- Each entry must be filled: $\Omega(n^2)$ time.
- That is, matrix multiplication must take at least quadratic time.
- Is this bound **tight**? Can it be increased?


Our theoretical lower bound
Our "naïve" algorithm
Θ(1)
Θ(n)
Θ(n²)
Θ(n³)

Strassen's Algorithm

- Cubic was as good as it got...
- ...until Strassen, 1969.
- Time complexity: $\Theta(n^{\log_2 7}) = \Theta(n^{2.8073})$

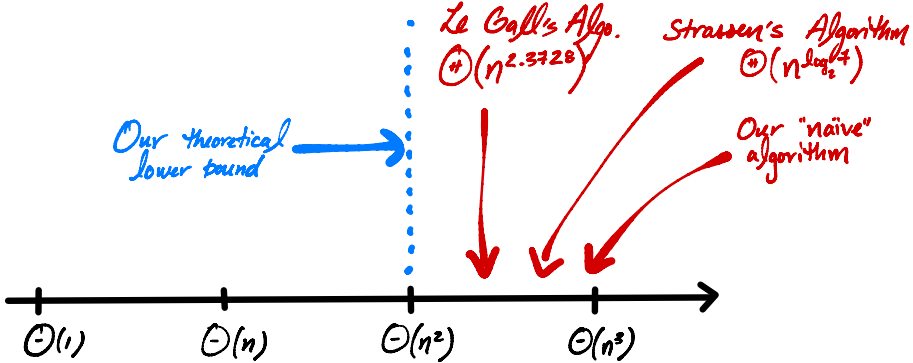

Strassen's Algorithm
$\Theta(n^{\log_2 7})$
Our "naïve" algorithm
Our theoretical lower bound
$\Theta(1)$
$\Theta(n)$
$\Theta(n^2)$
$\Theta(n^3)$


ω
3.0
2.9
2.8
2.7
2.6
2.5
2.4
naive
Strassen
Pan
Bini et al.
Schönhage
Romani
Coppersmith, Winograd
Strassen
Coppersmith, Winograd
Stothers
Le Gall
Williams
Year
1950
1960
1970
1980
1990
2000
2010
2015

Currently

- The fastest[1] known matrix multiplication algorithm is due to Le Gall.
- $\Theta(n^{2.3728639})$ time.

[1] In terms of asymptotic time complexity.


Le Gall's Algo.
$\Theta(n^{2.3728})$
Strassen's Algorithm
$\Theta(n^{\log_2 7})$
Our theoretical lower bound
Our "naïve" algorithm
$\Theta(1)$
$\Theta(n)$
$\Theta(n^2)$
$\Theta(n^3)$

Interestingly...

- No one knows what the lowest possible time complexity is.
- It could be $\Theta(n^2)$!
- The “best” matrix multiplication algorithm is probably still undiscovered.

Irony

- There are many matrix multiplication algorithms.
- How fast is numpy's matrix multiply?

Irony

- There are many matrix multiplication algorithms.
- How fast is numpy's matrix multiply?
- $\Theta(n^3)$.

Why?

- Strassen *et al.* have better asymptotic complexity.
- But much (much!) larger “hidden constants”.
- Remember, which is better for small n: $999{,}999n^2$ or n^3?

Optimization

- Numpy, most others use **highly optimized** cubic time algorithms[2]

[2]The constant c in $T(n) = cn^3 + \ldots$ is actually much less than 1, as can be verified by timing.

Main Idea

No one knows what the lowest possible time complexity of matrix multiplication is, and some algorithms are approaching $\Theta(n^2)$.

But most useful implementations take $\Theta(n^3)$ time.

DSC 40B

Theoretical Foundations II

Lecture 5 | Part 3

Searching a Database

Databases

- Large data sets are often stored in **databases**.

PID	FullName	Level
A1843	Wan Xuegang	SR
A8293	Deveron Greer	SR
A9821	Vinod Seth	FR
A8172	Aleix Bilbao	JR
A2882	Kayden Sutton	SO
A1829	Raghu Mahanta	FR
A9772	Cui Zemin	SR
⋮	⋮	⋮

Query

- What is the name of the student with PID `A8172`?

Linear Search

- We *could* answer this with a linear search.
- Recall worst-case time complexity: $\Theta(n)$.
- Is there a better way?

Theoretical Lower Bounds

- **Given**: an array `arr` and a target `t`, determine the index of `t` in the array.

- Lower bound: $\Omega(n)$
  - `linear_search` has the best possible worst-case complexity!

Theoretical Lower Bounds

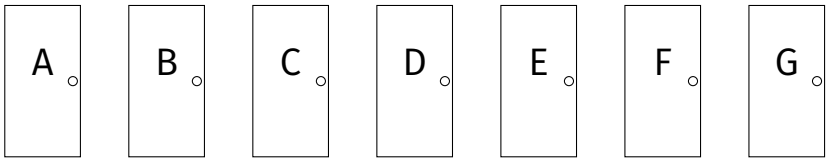
- **Given**: an **sorted** array `arr` and a target `t`, determine the index of `t` in the array.
- This is an **easier** problem.
- Lower bound: $\Omega(?)$

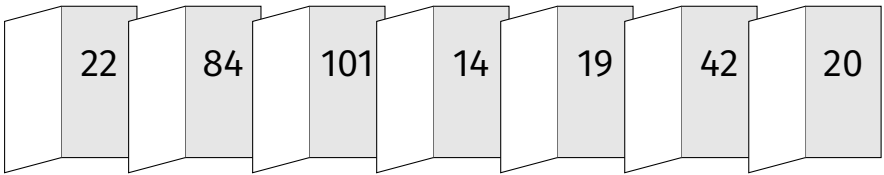
DSC 40B

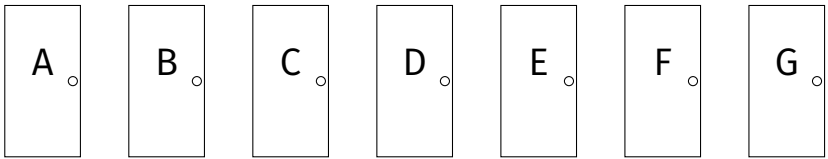
Theoretical Foundations II

Lecture 5 | Part 4

Binary Search


A
B
C
D
E
F
G


22
84
101
14
19
42
20


A
B
C
D
E
F
G

Game Show

- **Goal**: guess the door with number 42 behind it.
- **Caution**: with every wrong guess, your winnings are reduced.

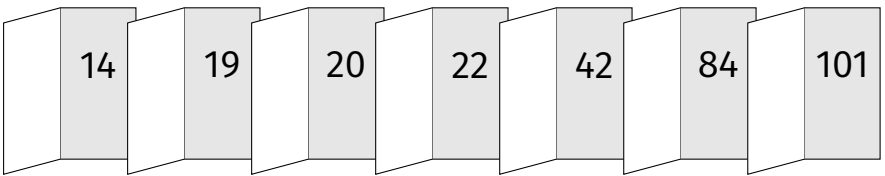
Strategy

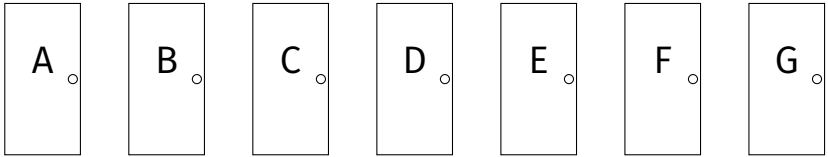
- Can't do much better than linear search.
  - "Is it door A?"
  - "OK, is it door B?"
  - "Door C?"

- After an incorrect first guess, the right door could be any of the other $n - 1$ doors!

But now...

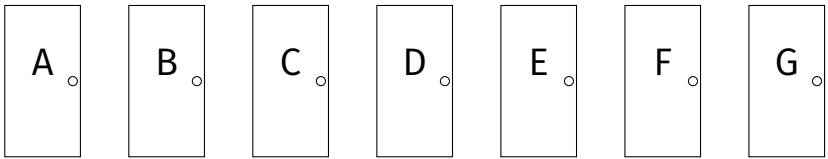
- Suppose the host tells you that the numbers are **sorted** in increasing order.

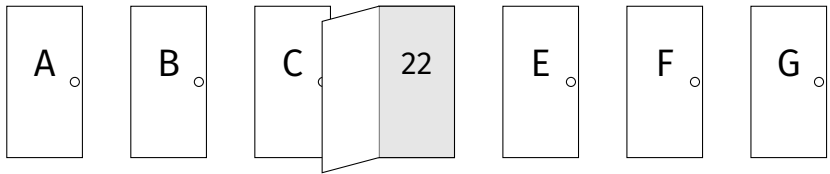


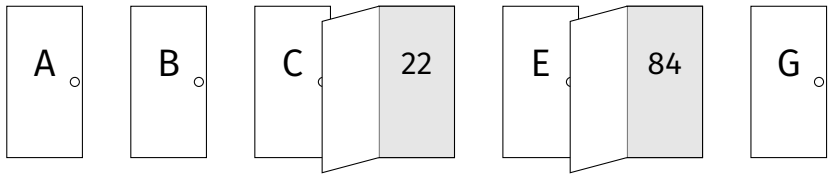


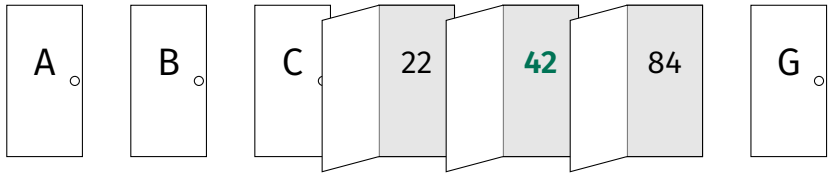
Exercise

Which door do you pick first?


A
B
C
D
E
F
G


A
B
C
22
E
F
G


A
B
C
22
E
84
G


A
B
C
22
42
84
G

Strategy

- First pick the middle door.
- Allows you to rule out half of the other doors.
- Pick door in the middle of what remains.
- Repeat, **recursively**.

Binary Search in Code

```
def binary_search(arr, t, start, stop):
    """
    Searches arr[start:stop] for t.
    Assumes arr is sorted.
    """
    if stop - start <= 0:
        return None
    middle = ___________ # index of the middle element
    if arr[middle] == t:
        return middle
    elif arr[middle] > t:
        return binary_search(arr, t, ______, ______)
    else:
        return binary_search(arr, t, ______, ______)
```

Exercise

Fill in the blanks:

```
def binary_search(arr, t, start, stop):
    """
    Searches arr[start:stop] for t.
    Assumes arr is sorted.
    """
    if stop - start <= 0:
        return None
    middle = ___________ # index of the middle element
    if arr[middle] == t:
        return middle
    elif arr[middle] > t:
        return binary_search(arr, t, ______, ______)
    else:
        return binary_search(arr, t, ______, ______)
```

The Middle Element

- What is the index of the middle element of `arr[start:stop]`?



Definition

The **floor** of a real number x, denoted $\lfloor x \rfloor$, is the *largest* integer that is $\leq x$.

Examples: $\lfloor 3.14 \rfloor = 3$ $\lfloor -4.5 \rfloor = -5$ $\lfloor 10 \rfloor = 10$

In LaTeX, $\lfloor x \rfloor$ is written: "`\lfloor x \rfloor`".

Definition

The **ceiling** of a real number x, denoted $\lceil x \rceil$, is the *smallest* integer that is $\geq x$.

Examples: $\lceil 3.14 \rceil = 4$ $\lceil -4.5 \rceil = -4$ $\lceil 10 \rceil = 10$

In LaTeX, $\lceil x \rceil$ is written: "`\lceil x \rceil`".

Binary Search

```
import math
def binary_search(arr, t, start, stop):
    """
    Searches arr[start:stop] for t.
    Assumes arr is sorted.
    """
    if stop - start <= 0:
        return None
    middle = math.floor((start + stop)/2)
    if arr[middle] == t:
        return middle
    elif arr[middle] > t:
        return binary_search(arr, t, start, middle)
    else:
        return binary_search(arr, t, middle+1, stop)
```

```python
import math
def binary_search(arr, t, start, stop):
    """
    Searches arr[start:stop] for t.
    Assumes arr is sorted.
    """
    if stop - start <= 0:
        return None
    middle = math.floor((start + stop)/2)
    if arr[middle] == t:
        return middle
    elif arr[middle] > t:
        return binary_search(arr, t, start, middle)
    else:
        return binary_search(arr, t, middle+1, stop)
```

t = 21

-10	-6	-3	1	2	5	12	21	33	35	42
0	1	2	3	4	5	6	7	8	9	10

Aside: Default Arguments

```
import math
def binary_search(arr, t, start=0, stop=None):
    if stop is None:
        stop = len(arr)
    if stop - start <= 0:
        return None
    middle = math.floor((start + stop)/2)
    if arr[middle] == t:
        return middle
    elif arr[middle] > t:
        return binary_search(arr, t, start, middle)
    else:
        return binary_search(arr, t, middle+1, stop)
```

Other Uses of Binary Search

- Binary search is a useful strategy in “real life”.

- Example: finding a bug in your code.
  - You have made 100 `git` commits (i.e., 100 changes).
  - You know the bug is introduced in one of them.
  - “Rewind” to commit #50.
    - If it bug is there, check in #25;
    - Otherwise, check in #75.
  - The `git bisect` command does this for you.

DSC 40B

Theoretical Foundations II

Lecture 5 | Part 5

Thinking Inductively

Recursion

- Recursive algorithms can almost look like **magic**.
- How can we be sure that `binary_search` works?

Tips

1. Make sure algorithm works in the **base case**.

2. Check that all recursive calls are on **smaller problems**.

3. **Assuming** that the recursive calls work, does the whole algorithm work?

Base Case

- Smallest input for which you can easily see that the algorithm works.
- Recursion works by making problem smaller until base case is reached.
- Usually $n = 0$ or $n = 1$ (or even both!)

Base Case: $n = 0$

- Suppose `arr[start:stop]` is empty.
- In this case, the function returns `None`.
  - **Correct!**

Base Case: $n = 1$

- Suppose `arr[start:stop]` has one element.
- If that element is the target, the algorithm will find it.
  - **Correct!**
- If it isn't, the algorithm will recurse on a problem of size 0 and return `None`.
  - **Correct!**

Recursive Calls

- Recursive calls must be on **smaller problems**.
  - Otherwise, base case never reached. Infinite recursion!

```python
import math
def binary_search(arr, t, start, stop):
    """
    Searches arr[start:stop] for t.
    Assumes arr is sorted.
    """
    if stop - start <= 0:
        return None
    middle = math.floor((start + stop)/2)
    if arr[middle] == t:
        return middle
    elif arr[middle] > t:
        return binary_search(arr, t, start, middle)
    else:
        return binary_search(arr, t, middle+1, stop)
```

```python
import math
def binary_search(arr, t, start, stop):
    """
    Searches arr[start:stop] for t.
    Assumes arr is sorted.
    """
    if stop - start <= 0:
        return None
    middle = math.floor((start + stop)/2)
    if arr[middle] == t:
        return middle
    elif arr[middle] > t:
        return binary_search(arr, t, start, middle)
    else:
        return binary_search(arr, t, middle+1, stop)
```

- Is `arr[start:middle]` smaller than `arr[start:stop]`?
- Is `arr[middle+1:stop]` smaller than `arr[start:stop]`?

Leap of Faith

- **Assume** the recursive calls work.
- Does the overall algorithm work, then?

```
import math
def binary_search(arr, t, start, stop):
    """
    Searches arr[start:stop] for t.
    Assumes arr is sorted.
    """
    if stop - start <= 0:
        return None
    middle = math.floor((start + stop)/2)
    if arr[middle] == t:
        return middle
    elif arr[middle] > t:
        return binary_search(arr, t, start, middle)
    else:
        return binary_search(arr, t, middle+1, stop)
```

Exercise

True or False: this code correctly computes the sum of `numbers`.

```
import math
def summation(numbers):
    n = len(numbers)
    if n == 0:
        return 0
    middle = math.floor(n / 2)
    return (
        summation(numbers[:middle])
        +
        summation(numbers[middle:])
    )
```

Induction

- These steps can be turned into a formal proof by **induction**.
- For us, less necessary to prove to other people.
- Instead, prove to **yourself** that your code works.
- We won't be doing formal inductive proofs.

Why does this work?

- Show that it works for size 1 (base case).
- $\implies$ will work for size 2 (inductive step).
- $\implies$ will work for sizes 3, 4 (inductive step).
- $\implies$ will work for sizes 5, 6, 7, 8 (inductive step).

DSC 40B

Theoretical Foundations II

Lecture 5 | Part 6

Recurrence Relations

Time Complexity of Binary Search

- What is the time complexity of `binary_search`?
- No loops!

Best Case

```
import math
def binary_search(arr, t, start, stop):
    """
    Searches arr[start:stop] for t.
    Assumes arr is sorted.
    """
    if stop - start <= 0:
        return None
    middle = math.floor((start + stop)/2)
    if arr[middle] == t:
        return middle
    elif arr[middle] > t:
        return binary_search(arr, t, start, middle)
    else:
        return binary_search(arr, t, middle+1, stop)
```

Worst Case

Let $T(n)$ be worst case time on input of size n.

```
import math
def binary_search(arr, t, start, stop):
    """
    Searches arr[start:stop] for t.
    Assumes arr is sorted.
    """
    if stop - start <= 0:
        return None
    middle = math.floor((start + stop)/2)
    if arr[middle] == t:
        return middle
    elif arr[middle] > t:
        return binary_search(arr, t, start, middle)
    else:
        return binary_search(arr, t, middle+1, stop)
```

Recurrence Relations

- We found

$$T(n) = \begin{cases} T(n/2) + \Theta(1), & n \geq 2 \\ \Theta(1), & n = 1 \end{cases}$$

- This is a **recurrence relation**.

Solving Recurrences

- We want simple, non-recursive formula for $T(n)$ so we can see how fast $T(n)$ grows.
  - Is it $\Theta(n)$? $\Theta(n^2)$? Something else?

- Obtaining a simple formula is called **solving** the recurrence.

Example: Getting Rich

- Suppose on day 1 of job, you are paid \$3.
- Each day thereafter, your pay is doubled.
- Let $S(n)$ be your pay on day n:

$$S(n) = \begin{cases} 2 \cdot S(n-1), & n \geq 2 \\ 3, & n = 1 \end{cases}$$

Example: Unrolling

$$S(n) = \begin{cases} 2 \cdot S(n-1), & n \geq 2 \\ 3, & n = 1 \end{cases}$$

- Take $n = 4$.

Solving Recurrences

We'll use a four-step process to solve recurrences:

1. "Unroll" several times to find a pattern.
2. Write general formula for *k*th unroll.
3. Solve for # of unrolls needed to reach base case.
4. Plug this number into general formula.

Step 1: Unroll several times

$$S(n) = \begin{cases} 2 \cdot S(n-1), & n \geq 2 \\ 3, & n = 1 \end{cases}$$

Step 2: Find general formula

$$
\begin{aligned}
S(n) &= 2 \cdot S(n-1) \\
&= 2 \cdot 2 \cdot S(n-2) \\
&= 2 \cdot 2 \cdot 2 \cdot S(n-3)
\end{aligned}
$$

On step k:

Step 3: Find step # of base case

- On step k, $S(n) = 2^k \cdot S(n - k)$.
- When do we see $S(1)$?

Step 4: Plug into general formula

- From step 2: $S(n) = 2^k \cdot S(n - k)$.
- From step 3: Base case of $S(1)$ reached when $k = n - 1$.
- So:

Solving the Recurrence

- We have **solved** the recurrence[3]:

$$S(n) = 3 \cdot 2^{n-1}$$

- This is the **exact** solution. The **asymptotic** solution is $S(n) = \Theta(2^n)$.

- We'll call this method "solving by unrolling".

[3] On day 20, you'll be paid ≈1.5 million dollars.

DSC 40B

Theoretical Foundations II

Lecture 5 | Part 7

Binary Search Recurrence

Binary Search

- What is the time complexity of `binary_search`?
- Best case: $\Theta(1)$.
- Worst case:

$$T(n) = \begin{cases} T(n/2) + \Theta(1), & n \geq 2 \\ \Theta(1), & n = 1 \end{cases}$$

Simplification

- When solving, we can replace $\Theta(f(n))$ with $f(n)$:

$$T(n) = \begin{cases} T(n/2) + 1, & n \geq 2 \\ 1, & n = 1 \end{cases}$$

- As long as we state final answer using Θ notation!

Another Simplification

- When solving, we can assume n is a power of 2.

Step 1: Unroll several times

$$T(n) = \begin{cases} T(n/2) + 1, & n \geq 2 \\ 1, & n = 1 \end{cases}$$

Step 2: Find general formula

$$
\begin{aligned}
T(n) &= T(n/2) + 1 \\
&= T(n/4) + 2 \\
&= T(n/8) + 3
\end{aligned}
$$

On step k:

Step 3: Find step # of base case

- On step k, $T(n) = T(n/2^k) + k$
- When do we see $T(1)$?

Step 4: Plug into general formula

- $T(n) = T(n/2^k) + k$
- Base case of $T(1)$ reached when $k = \log_2 n$.
- So:

Note

- Remember: $\log_b x = (\log_a x)/(\log_a b)$
- So we don't write $\Theta(\log_2 n)$
- Instead, just: $\Theta(\log n)$

Time Complexity of Binary Search

- Best case: $\Theta(1)$
- Worst case: $\Theta(\log n)$

Is binary search fast?

- Suppose all 10^{19} grains of sand are assigned a unique number, sorted from least to greatest.
- Goal: find a particular grain.
- Assume one basic operation takes 1 nanosecond.

Is binary search fast?

- Suppose all 10^{19} grains of sand are assigned a unique number, sorted from least to greatest.
- Goal: find a particular grain.
- Assume one basic operation takes 1 nanosecond.
- Linear search: 317 years.

Is binary search fast?

- Suppose all 10^{19} grains of sand are assigned a unique number, sorted from least to greatest.
- Goal: find a particular grain.
- Assume one basic operation takes 1 nanosecond.
- Linear search: 317 years.
- Binary search: ≈ 60 nanoseconds.

Exercise

Binary search seems so much faster than linear search. What's the caveat?

Caveat

- The array must be **sorted**.
- This takes $\Omega(n)$ time.

Why use binary search?

- If data is **not sorted**, sorting + binary search takes longer than linear search.
- But if doing **multiple queries**, looking for nearby elements, sort once and use binary search after.

Theoretical Lower Bounds

- A tight lower bound for searching a sorted list is $\Omega(\log n)$.
- This means that binary search has **optimal** worst case time complexity.

Databases

- Some database servers will **sort** by key, use binary search for queries.
- Often instead of sorting, **B-Tree indexes** are used.
- But sorting + binary search still used when space is limited.